PONTIFICIA UNIVERSIDAD CATÓLICA MADRE Y MAESTRA
Facultad de Ciencias e Ingeniería
Escuela de Ingeniería en Computación y Telecomunicaciones

DESARROLLO DE APLICACIONES MÓVILES

DOCUMENTO DE ARQUITECTURA DE SOFTWARE

RegiDex

Offline-First, persistencia y sincronización, integración GraphQL, conectividad, sistema de diseño Regi y decisiones de arquitectura (ADR)

INTEGRANTES
Randy A. Germosén
Rafael J. Ramírez

DOCENTE
Freddy Peña

VERSIÓN
1.0

FECHA
23 de octubre de 2026

1. Introducción

Este documento describe **cómo** construimos RegiDex para cumplir la especificación de requisitos (`docs/SRS_RegiDex.pdf`). Los diagramas reflejan nuestra solución concreta, no los de referencia del enunciado. La tabla siguiente relaciona ambos.

ENUNCIADO	NUESTRO DIAGRAMA	TEMA
D1 Flujo Online/Offline	D05	Arranque, verificación, sync y reintento
D2 Secuencia de consulta	D06	UI → estado → repositorio → BD ← sync ← API
D3 Máquinas de estados	D07, D08	Disponibilidad del servicio y sincronización
D4 Arquitectura general	D03	Capas y componentes
D5 Estructura del proyecto	D04	Carpetas de `lib/`, incluida `lib/ui/`
D6 Modelo de datos	D09–D12	Catálogo, detalle, usuario/sync y Champions
—	D01, D02, D13–D18, M01, M02	Contexto, casos de uso, sync paginada, integridad, imágenes, Team Builder, navegación, cronograma y mockups

1.1 Principios y cómo los aplicamos

PRINCIPIO	APLICACIÓN EN REGIDEX
P-01 Fuente de verdad local	Los widgets solo consumen streams de Drift; las respuestas remotas se validan y escriben antes de verse.
P-02 Online-Preferred	Si hay servicio y el dato está vencido, sincronizamos en segundo plano sin bloquear.
P-03 Separación de responsabilidades	Features con `presentation/domain/data`; lo transversal en `core/`; lo visual en `ui/`.
P-04 Regla de dependencia	`domain` es Dart puro, sin Flutter, GraphQL ni Drift.
P-05 Degradación segura	Un fallo remoto nunca borra datos válidos; la UI sigue con lo local y avisa.
P-06 Transparencia del estado	El LED muestra el servicio; la segunda línea muestra el origen y la hora de los datos.
P-07 Verificabilidad	Once ADR (§11), escenarios PA reproducibles, simulador de caída e inspector de BD.

2. Contexto, capas y estructura

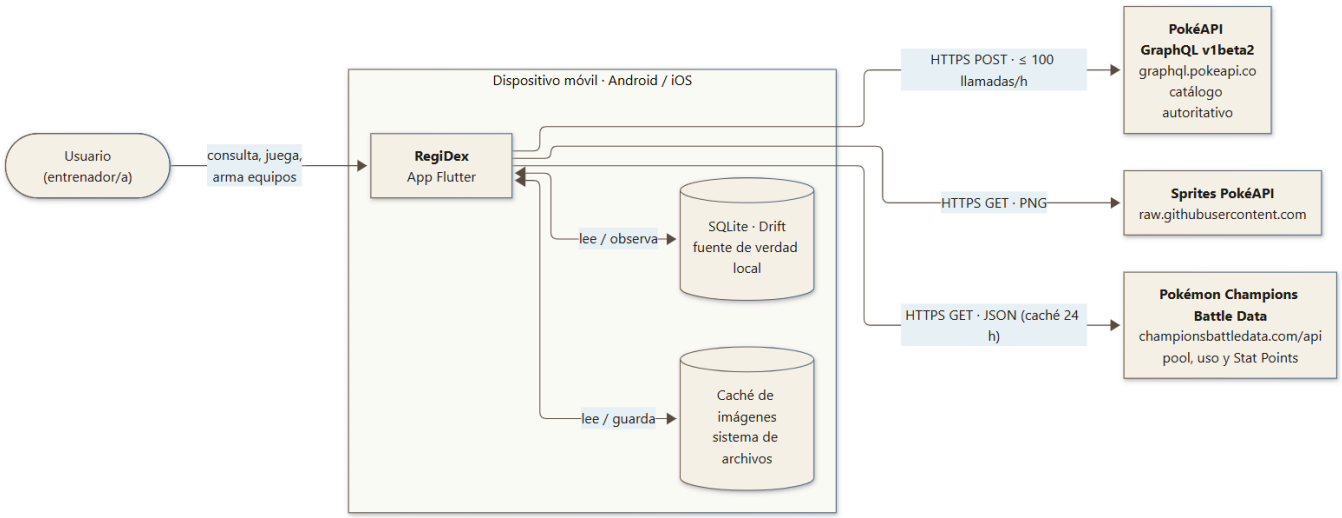


Figura 1. Contexto del sistema (D01).

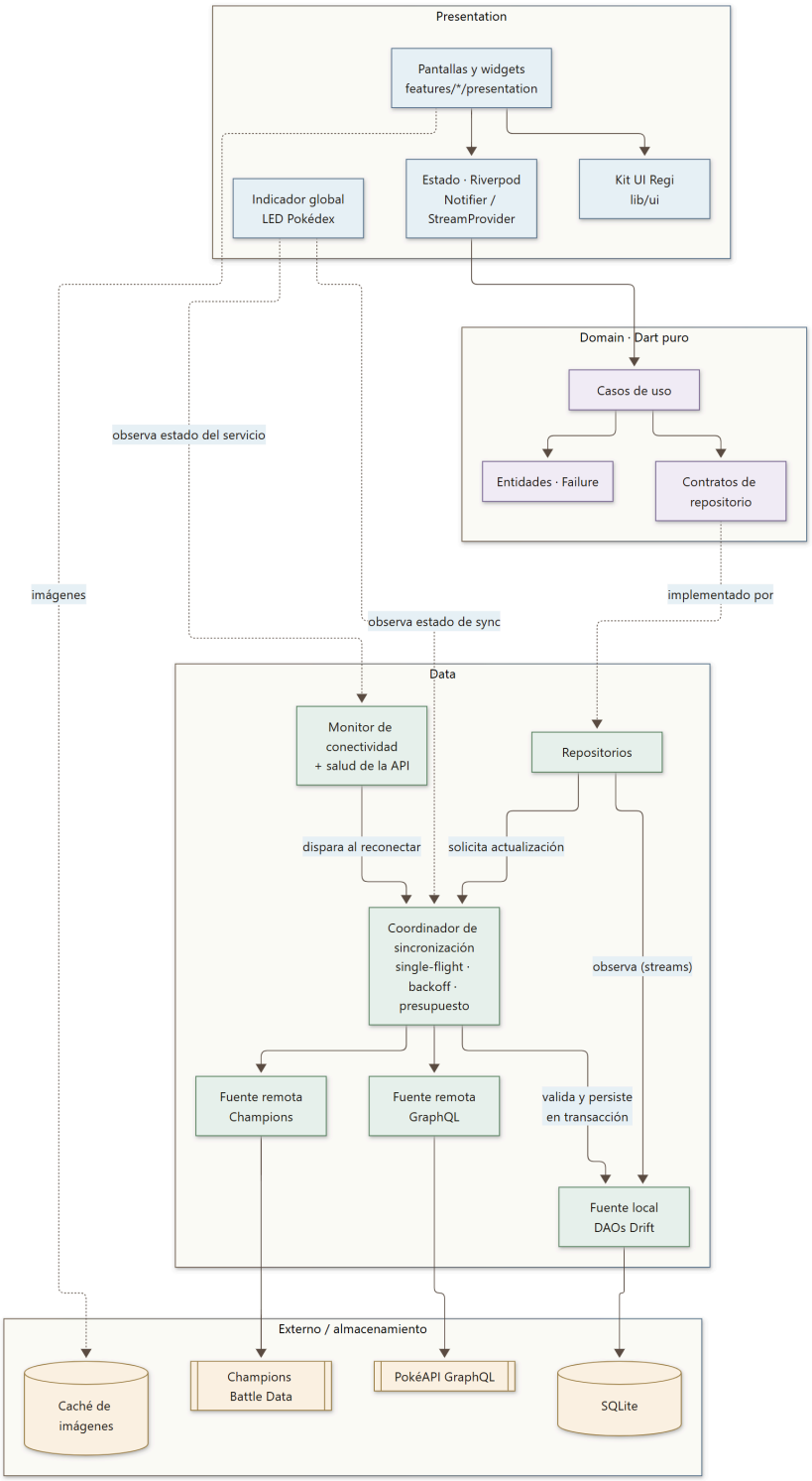


Figura 2. Arquitectura general (D03). Todas las flechas apuntan hacia abajo: la presentación depende del dominio y los datos implementan sus contratos.

CAPA	CONTIENE	NO PUEDE
Presentation	Páginas, widgets, providers de estado (`AsyncValue` y estados sellados), indicador global y el kit `lib/ui` .	Importar el cliente GraphQL, Drift ni DTOs.

CAPA	CONTIENE	NO PUEDE
Domain	Entidades inmutables, casos de uso, contratos de repositorio y `Failure`.	Depender de Flutter o de infraestructura.
Data	Repositorios, fuentes local y remotas, DTOs con mapeadores, `SyncCoordinator`, `IntegrityChecker` y `ConnectivityMonitor`.	Exponer DTOs ni excepciones de infraestructura.

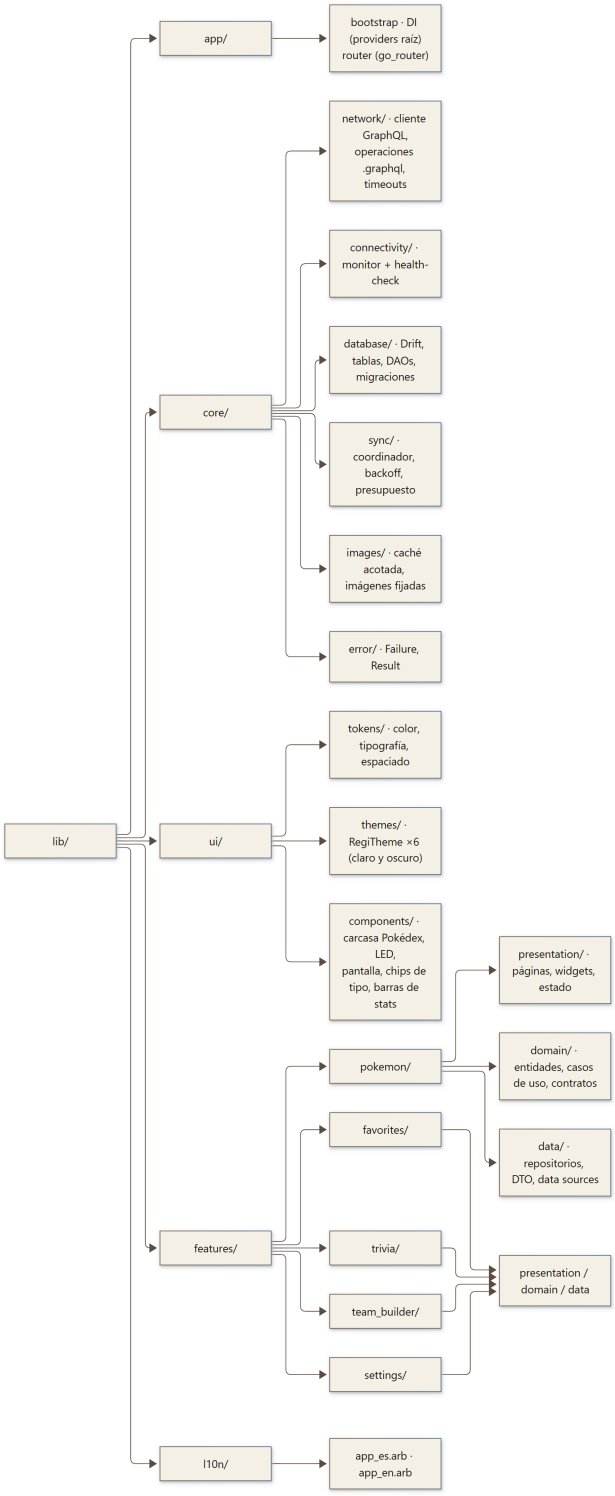


Figura 3. Estructura de `lib/` por funcionalidades y capas (D04).

El directorio `lib/ui/` es nuestro sistema de diseño. Contiene:

- `tokens/`: colores base, tipografía, espaciado y radios.
- `themes/`: `RegiTheme` como `ThemeExtension` y los seis temas, cada uno en claro y oscuro.
- `components/`: `PokedexShell` (carcasa), `StatusLed`, `DexScreen` (pantalla con bisel), `TypeChip`, `StatBar`, `EmptyState` y `ErrorState`.

Las features componen estos widgets y nunca definen colores sueltos.

3. Offline-First

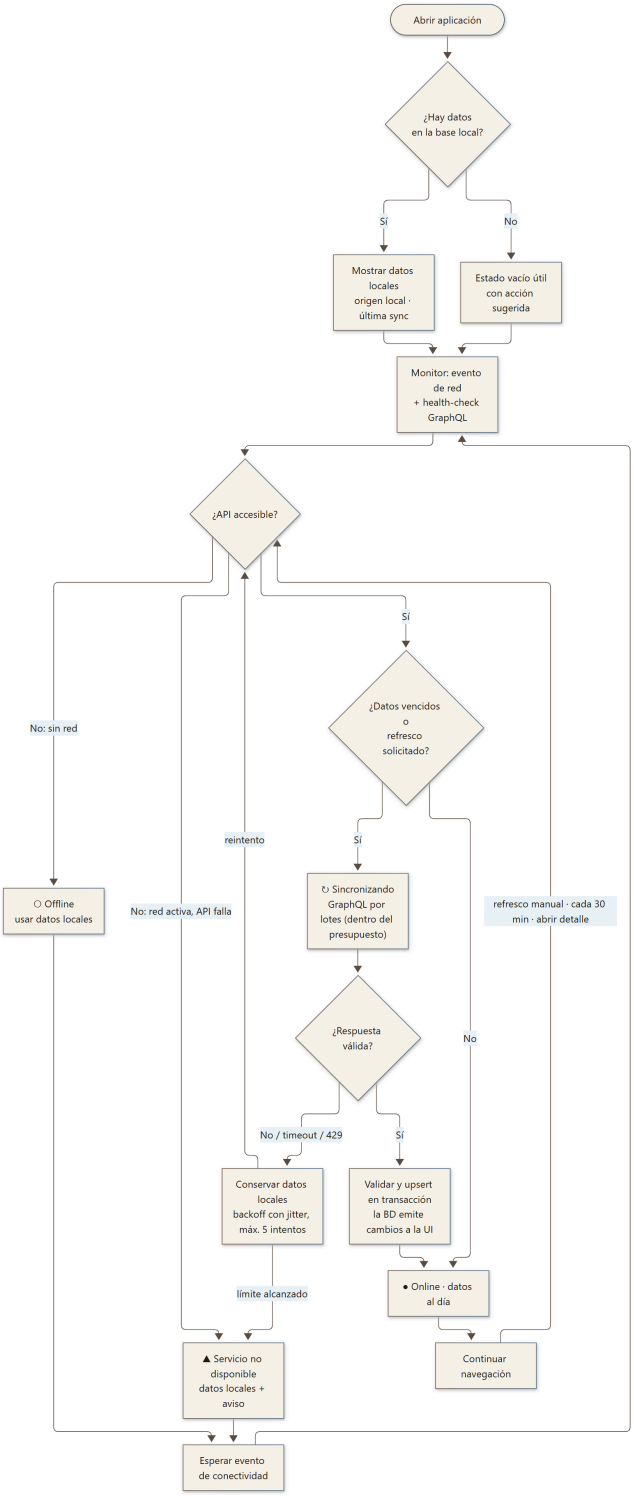


Figura 4. Flujo desde el arranque hasta la reconexión (D05).

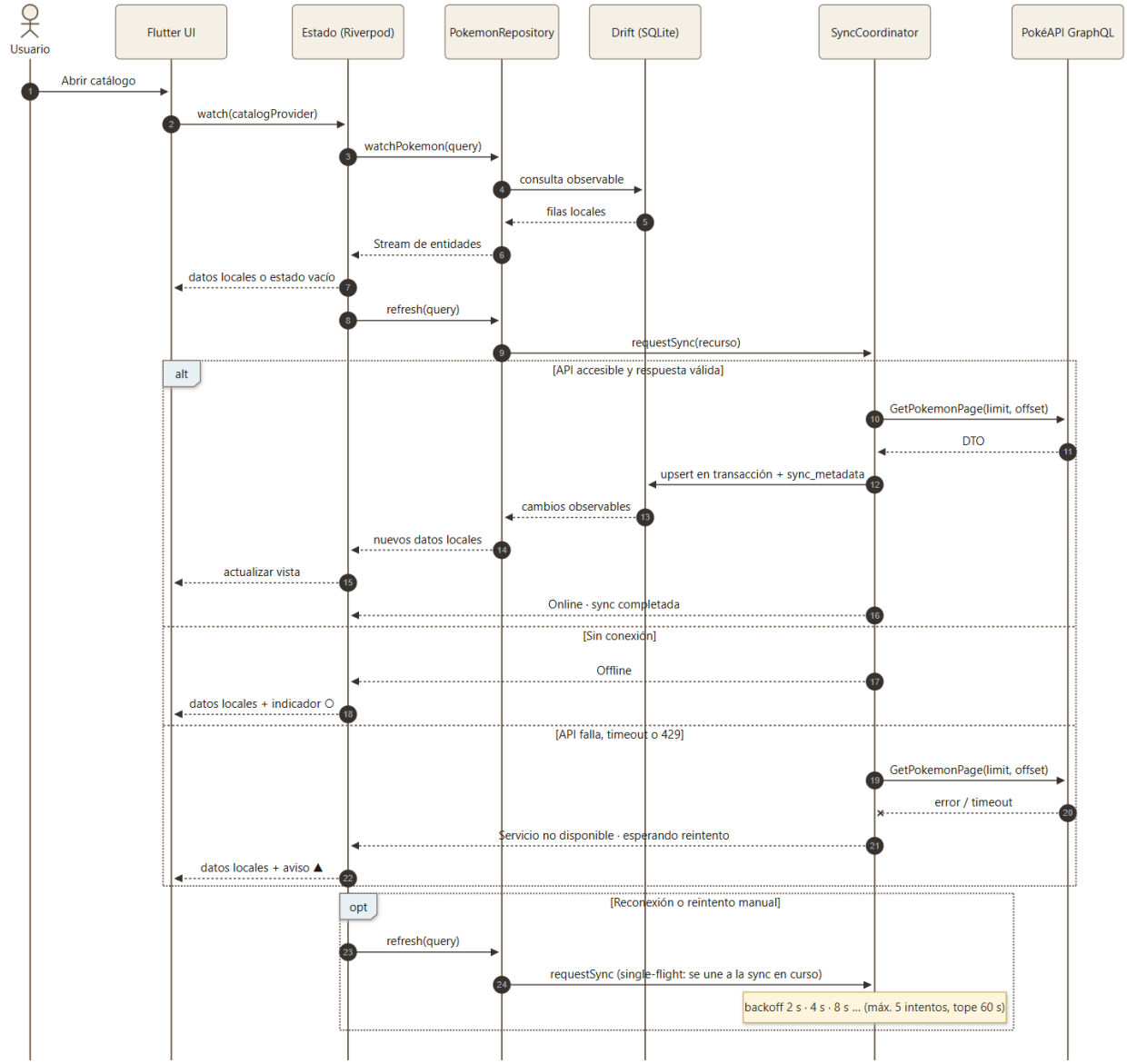


Figura 5. Secuencia de consulta y recuperación (D06). La UI mantiene la suscripción a la BD durante todo el proceso.

ASPECTO	REGIDEX (OFFLINE-FIRST)	LO QUE EVITAMOS (CACHÉ HTTP)
Fuente de la UI	Streams de Drift.	Respuesta de red o caché opaca.
Modelo	Tablas normalizadas con índices (D09–D12).	Blobs de respuesta por URL.
Consultas nuevas offline	Búsqueda, filtros y orden por SQL.	Solo solicitudes idénticas a las cacheadas.
Datos del usuario	Mismo esquema, tablas separadas.	Fuera de la caché.
Vigencia	`sync_metadata` por recurso con política explícita.	TTL o cabeceras.
Reconexión	Coordinada, single-flight, backoff y presupuesto.	Reintentos sin control.

El cliente GraphQL se configura con `FetchPolicy.noCache`: su caché normalizada no aporta y duplicaría memoria.

4. Conectividad e indicador global

4.1 Modelo de estados

Separamos dos dimensiones independientes y las componemos en el indicador junto con el origen y la frescura de los datos:

- `ServiceStatus` ∈ {verificando, online, offline, servicioNoDisponible}
- `SyncStatus` ∈ {inactiva, sincronizando, completada, fallida, esperandoReintento}

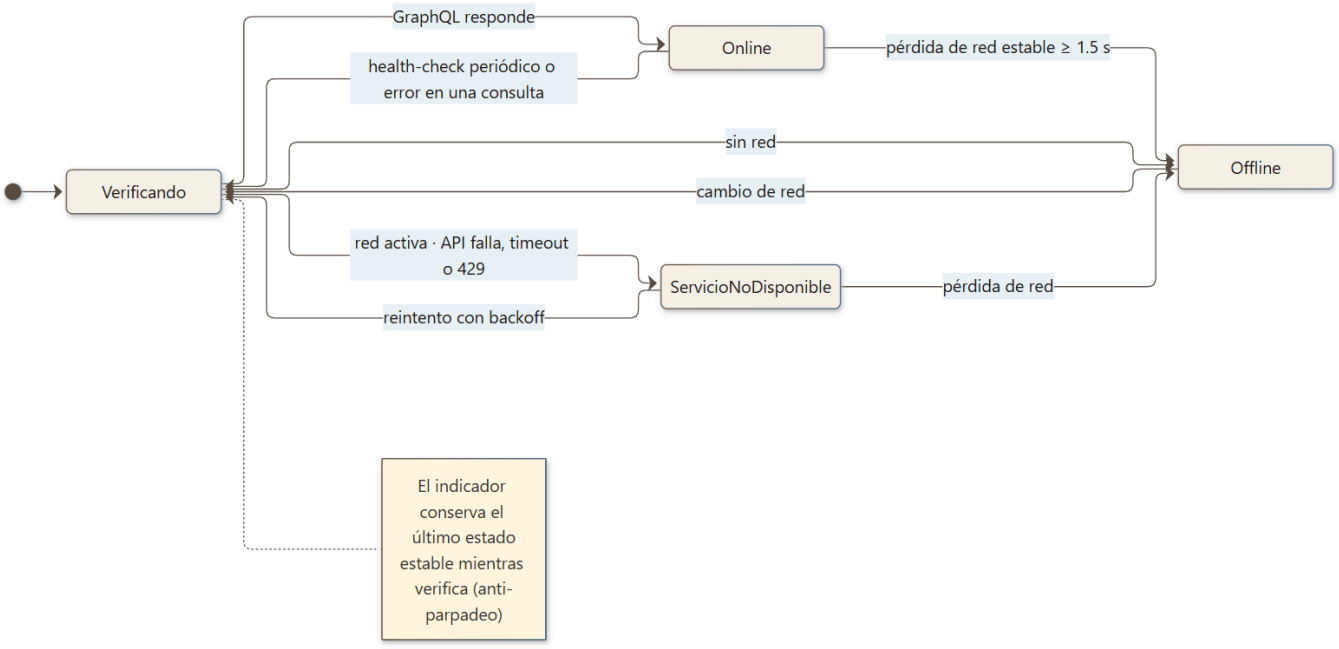


Figura 6. Máquina de estados de disponibilidad (D07).

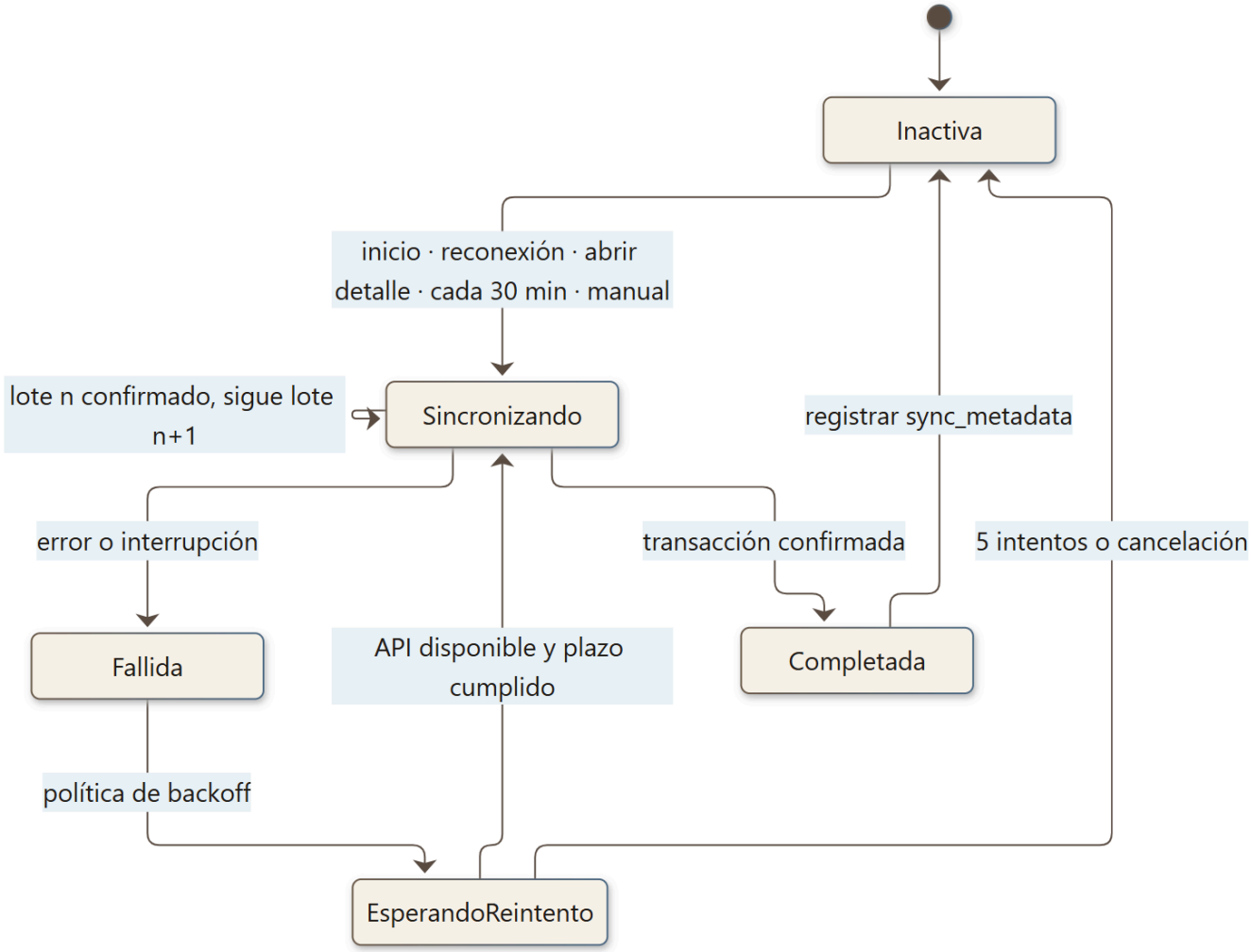


Figura 7. Máquina de estados de sincronización (D08).

Cómo detectamos el servicio.

- `connectivity_plus` avisa de cambios de red, pero solo como *pista*.
- La verdad la da `HealthCheck`: la consulta `pokemon(limit: 1) { id }` (31 bytes de respuesta) con timeout de 5 s.
- Para no gastar el presupuesto de 100 llamadas por hora:
  - Toda consulta real exitosa cuenta como health-check.
  - El ping explícito solo se envía al cambiar la red, al salir de `servicioNoDisponible` (con backoff) y como máximo cada 10 minutos si no hubo tráfico.

Clasificación de fallos.

SITUACIÓN	ESTADO DEL SERVICIO
Sin interfaz de red	Offline
Red presente con DNS/socket fallido, timeout, HTTP 5xx, 429 o error GraphQL de esquema	Servicio no disponible

Anti-parpadeo (IND-06). Un estado nuevo debe sostenerse **1.5 s** antes de mostrarse. Mientras se verifica, el LED conserva el último estado estable.

4.2 El LED de la Pokédex



Indicador global (IND-01…07). El lente de la Pokédex es el LED de estado: el color permite reconocerlo de un vistazo, pero cada estado tiene además un **glifo propio** (● ○ ▲ ↻), una **etiqueta de texto** con el origen y la vigencia de los datos y una etiqueta **Semantics** para lectores de pantalla. Al tocar el LED se abre un popover con el detalle y la acción «Sincronizar ahora». Tema mostrado: Regirock claro. Las siluetas son marcadores de posición.

Figura 8. LED en sus cuatro estados (M01).

ESTADO	COLOR	GLIFO	LÍNEA 1	LÍNEA 2 (ORIGEN · VIGENCIA)
Online	Verde `#1E9E4A`	●	En línea	Datos remotos · actualizado HH:mm
Offline	Gris `#6B7280`	○	Sin conexión	Datos locales · última sync HH:mm
Servicio no disponible	Ámbar `#F2A900`	▲	Servicio no disponible	Red activa, API sin respuesta · datos locales
Sincronizando	Azul `#2F6FD6`	↻	Sincronizando	En línea · puedes seguir navegando (barra fina)

Accesibilidad.

- El LED es un botón de 48 dp con `Semantics(label: 'Estado de conexión: …, datos locales de las 09:15', button: true)`.
- Tocarlo abre un **popover** con red, PokéAPI, datos y próximo reintento, más la acción «Sincronizar ahora».
- Con *reducir movimiento* activado se desactiva el anillo animado.

4.3 Herramientas para la demostración

NECESIDAD	MECANISMO
PA-08: API caída con red activa	**Ajustes → Depuración → Simular API caída**. El provider del endpoint cambia a `https://graphql.pokeapi.co/__regidex_down__`, que responde con error, mientras la red sigue activa. Solo existe en builds de depuración y perfil.
PA-13 / PA-16: inspeccionar la BD	Database Inspector de Android Studio (App Inspection) sobre `regidex.sqlite`. En la consola registramos cada operación GraphQL con su duración y su intento.
PA-09: interrumpir la sync	Modo avión o cierre forzado durante la carga inicial; luego se compara `sync_metadata` antes y después.

5. Persistencia y sincronización

5.1 Modelo de datos local

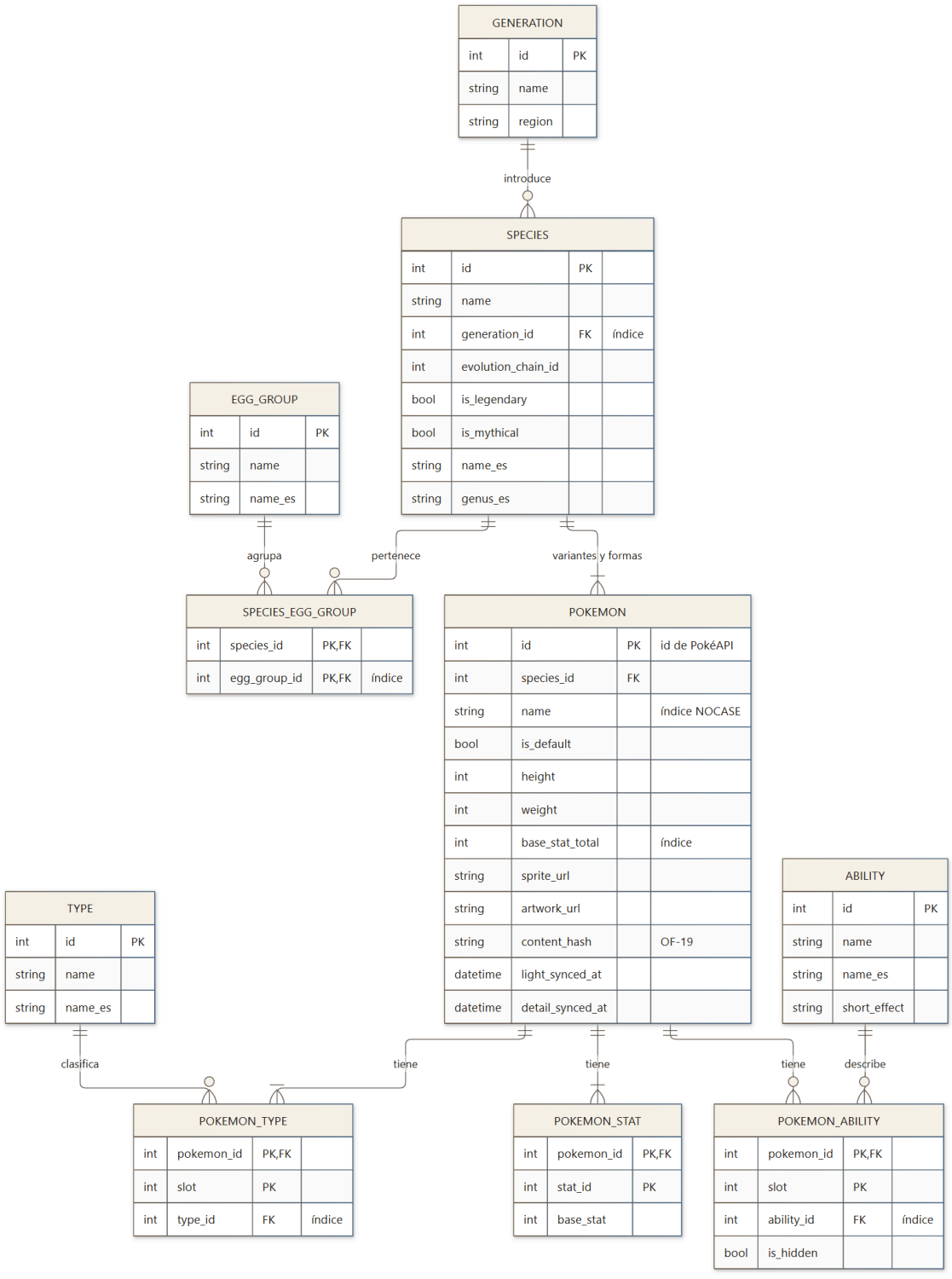


Figura 9. Catálogo: identidad, tipos, habilidades, stats, especie, generación y grupos huevo (D09).

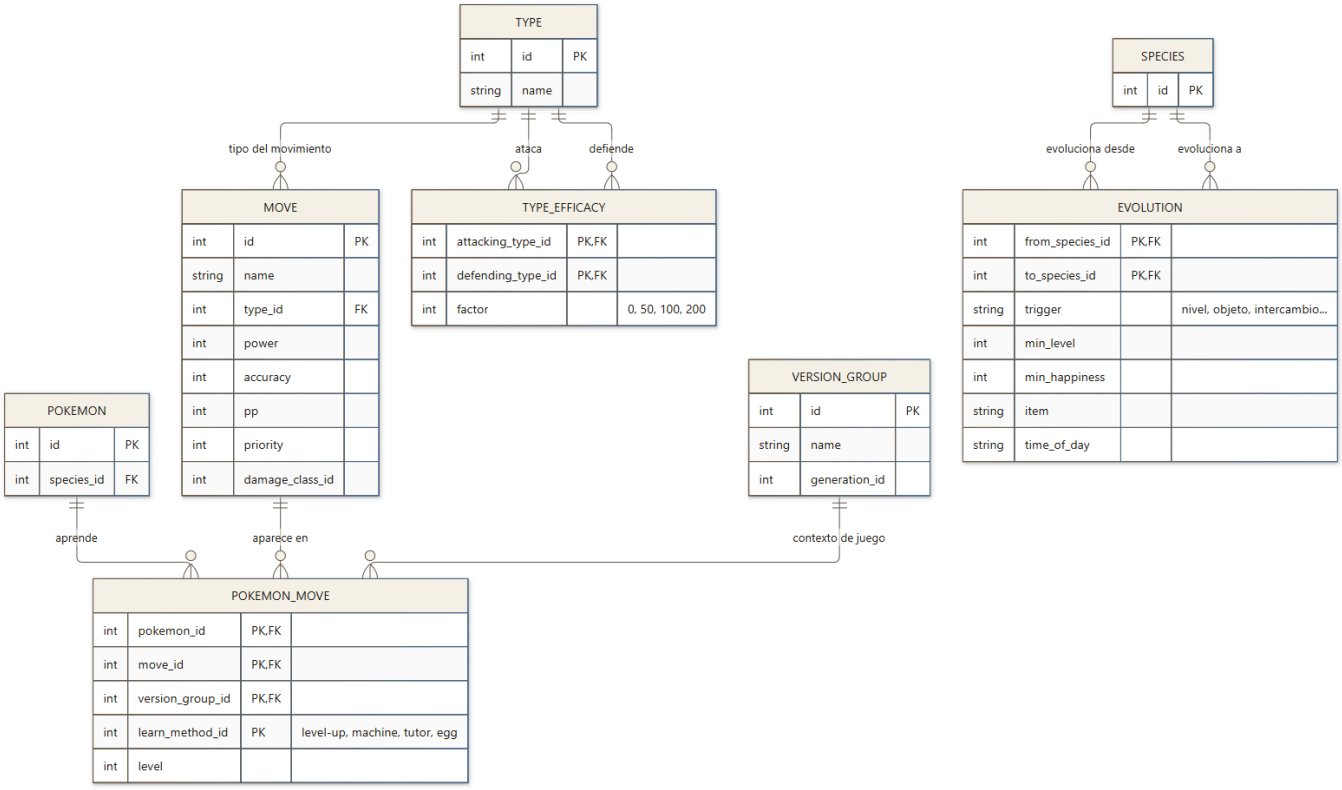


Figura 10. Movimientos, evoluciones y eficacias (D10).

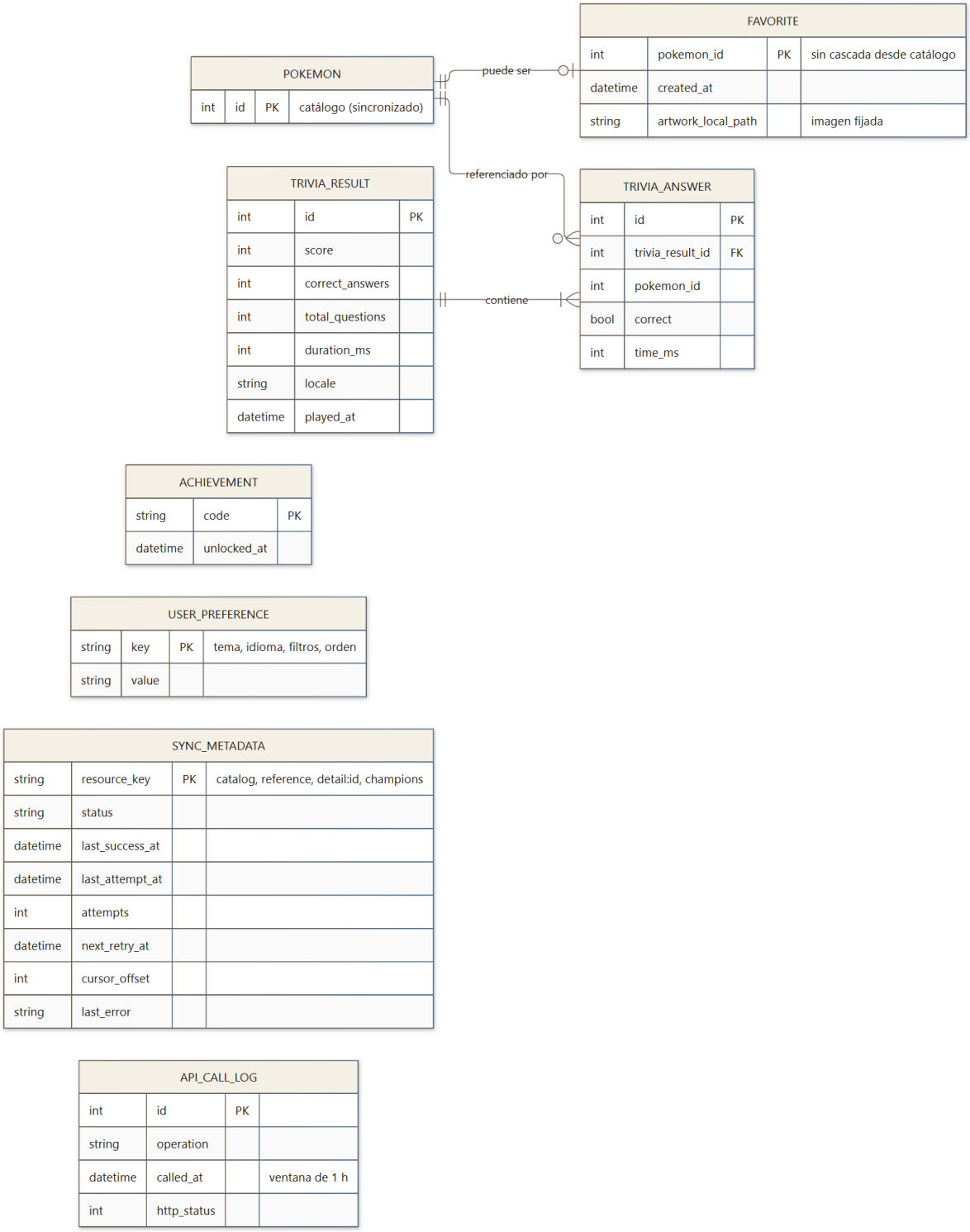


Figura 11. Datos del usuario (favoritos, trivia, logros y preferencias) y metadatos de sincronización (D11).

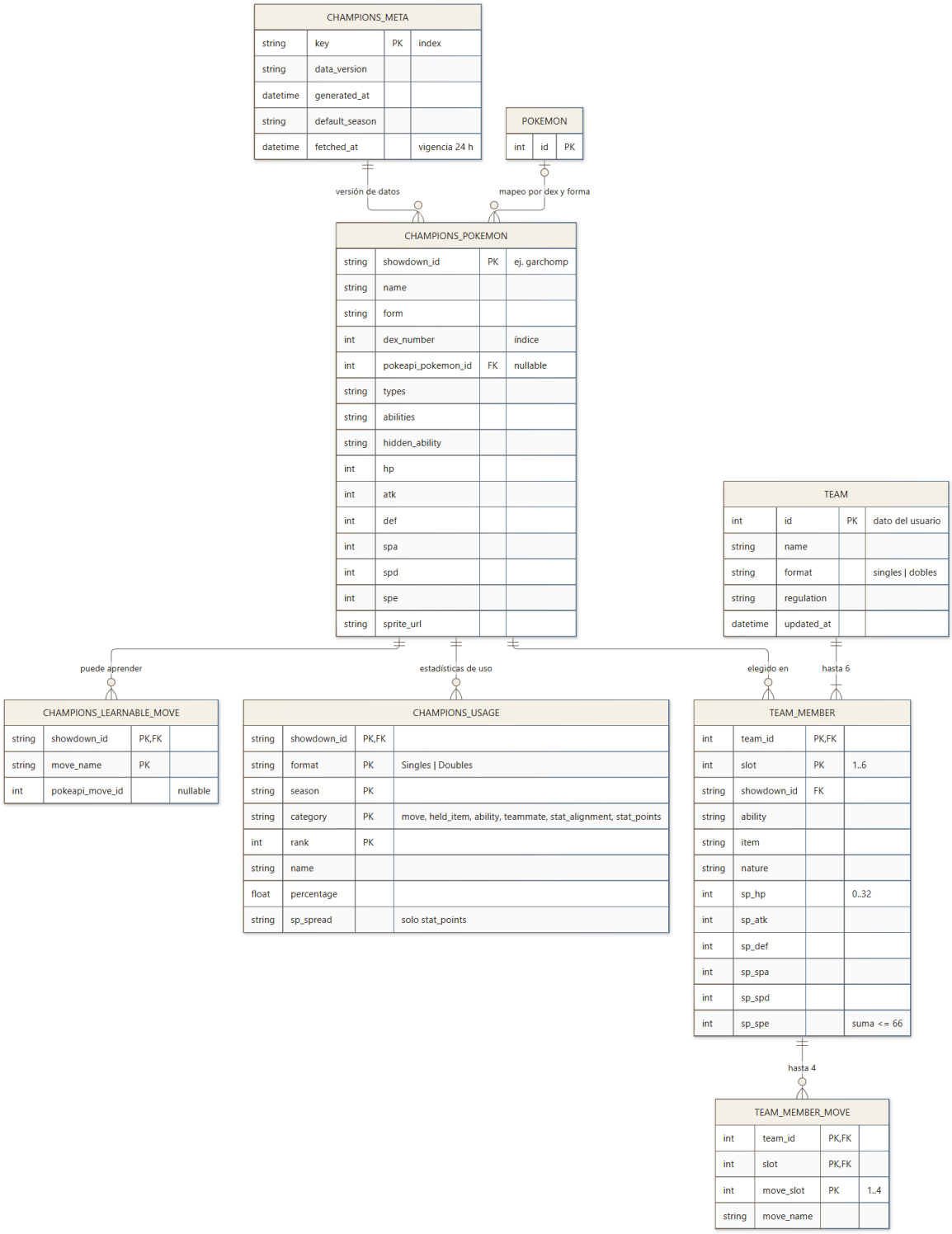


Figura 12. Caché de Pokémon Champions Battle Data y equipos del usuario (D12).

5.2 Decisiones PS-01 a PS-12

CÓDIGO	DECISIÓN
PS-01 Modelo	SQLite con Drift. Claves naturales de PokéAPI; tablas puente con PK compuestas. Índices en `pokemon(name COLLATE NOCASE)`, `pokemon(base_stat_total)`, `species(generation_id)`, `pokemon_type(type_id)`, `pokemon_ability(ability_id)` y `species_egg_group(egg_group_id)`. Todas las variantes de filtro se resuelven con `EXISTS` sobre las tablas puente.
PS-02 Carga inicial	**No bloqueante**. (1) `GetReferenceData`, 1 llamada (~147 KB): tipos, generaciones, grupos huevo, eficacias, version groups, 314 habilidades y 937 movimientos. (2) Catálogo ligero completo en **6 lotes de 200** (~165 KB c/u, 1025 Pokémon). (3) Detalle y artwork bajo demanda. Esto deja los filtros listos offline con apenas 7 llamadas.
PS-03 Vigencia	Referencia: 30 días. Catálogo ligero: 7 días. Detalle: 3 días. Champions: 24 h. Fuera de ese plazo el dato se muestra igual, marcado como «vencido», y se refresca si hay servicio y presupuesto.
PS-04 Disparadores	Inicio, reconexión, apertura de detalle, cada 30 min en primer plano mientras haya servicio, pull-to-refresh y «Sincronizar ahora». No hay sync en segundo plano del sistema operativo: iOS no la garantiza y no la necesitamos.
PS-05 Identidad	`pokemon.id`, `species.id`, `type.id`, `ability.id`, `move.id`, `egg_group.id` y `version_group.id` de PokéAPI. Champions usa `showdown_id`.
PS-06 Duplicados	`insertOnConflictUpdate` por PK. Las relaciones de un Pokémon se reemplazan dentro de la misma transacción (borrar e insertar las suyas). PA-13 lo verifica con conteos.
PS-07 Sync incompleta	Cada lote es **una transacción** que incluye el avance de `sync_metadata.cursor_offset`. Si algo se interrumpe, el lote hace rollback y la siguiente sync reanuda desde el último offset confirmado. El detalle es todo o nada.
PS-08 Imágenes	Fuera de la BD (ADR-004). Caché de archivos LRU de 150 MB y 1500 archivos con `flutter_cache_manager`. Sprites para listas, artwork solo en el detalle, y artwork de favoritos fijado en `ApplicationDocuments/pinned/`. Ajustes ofrece «Borrar caché de imágenes».
PS-09 Última sync	`sync_metadata` por recurso (`reference`, `catalog`, `detail:{id}`, `champions`). El indicador muestra la hora del recurso que se está viendo.
PS-10 Recuperación	Backoff $2\text{ s} \times 2^n$ con jitter de ±20 %, tope de 60 s y 5 intentos. Estado `failed` persistido con `last_error`, `attempts` y `next_retry_at`. Migraciones Drift con `schemaVersion` y pruebas de migración.
PS-11 Datos de usuario	Las tablas `favorite`, `trivia_*`, `achievement`, `team*` y `user_preference` no tienen FK en cascada hacia el catálogo. La sync no tiene código que escriba en ellas.
PS-12 Alteraciones	Detalle en §5.4.

5.3 Sincronización paginada

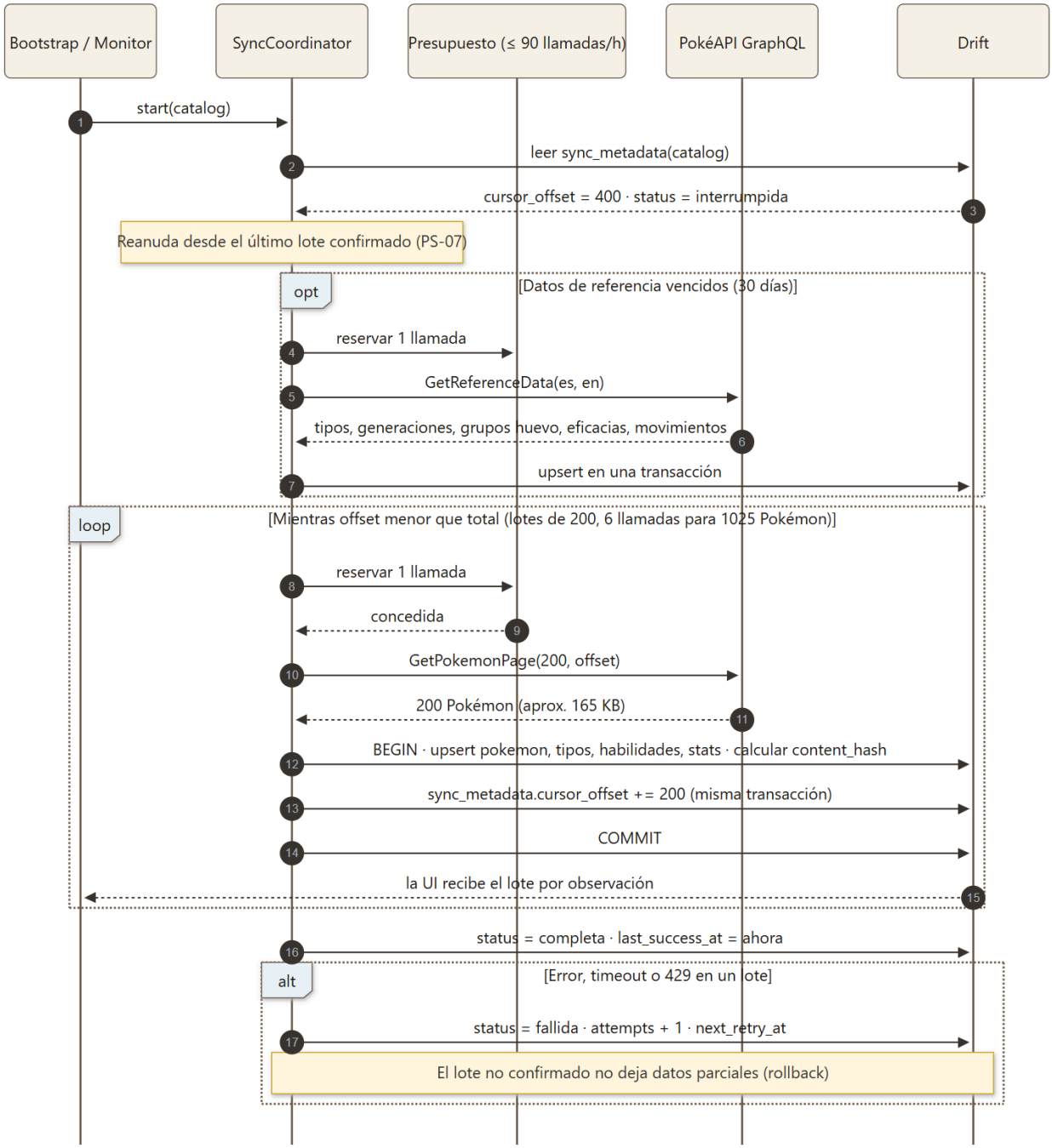


Figura 13. Carga inicial por lotes con presupuesto y reanudación (D13).

Concurrencia.

- `SyncCoordinator` mantiene un `Map<String, Future>` por `resource_key` (single-flight): una segunda solicitud del mismo recurso recibe el mismo `Future`.
- Los lotes se procesan en serie para respetar el presupuesto.
- El mapeo de DTO a filas corre en `Isolate.run`.

5.4 Detección y restauración de datos alterados (OF-19/20/21)

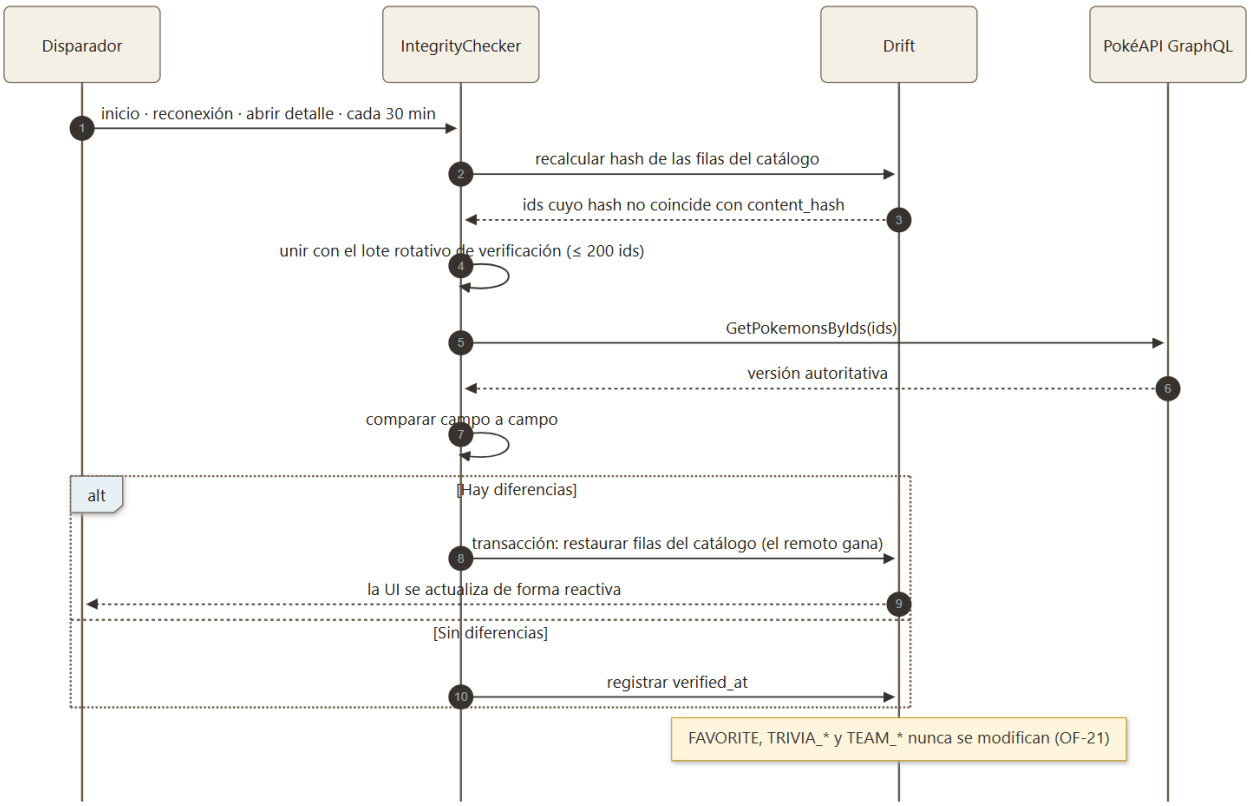


Figura 14. Verificación y restauración (D14).

Al persistir cada fila del catálogo calculamos `content_hash = SHA-1` de sus campos canónicos (nombre, medidas, tipos, habilidades y stats, ordenados). La detección tiene dos niveles:

1. **Local, sin red y barato.** Se recalcula el hash y se compara con `content_hash`. Si alguien edita la fila con el inspector, no coincide y el id queda marcado para restaurar.
2. **Contra el remoto.** En cada disparador (OF-20) verificamos un lote rotativo de hasta 200 ids con `GetPokemonsByIds`, más los marcados. Al abrir un detalle siempre comparamos con el remoto. Si hay diferencias, se restaura en transacción: **el remoto siempre gana**.

Las tablas de usuario no participan (OF-21).

5.5 Imágenes

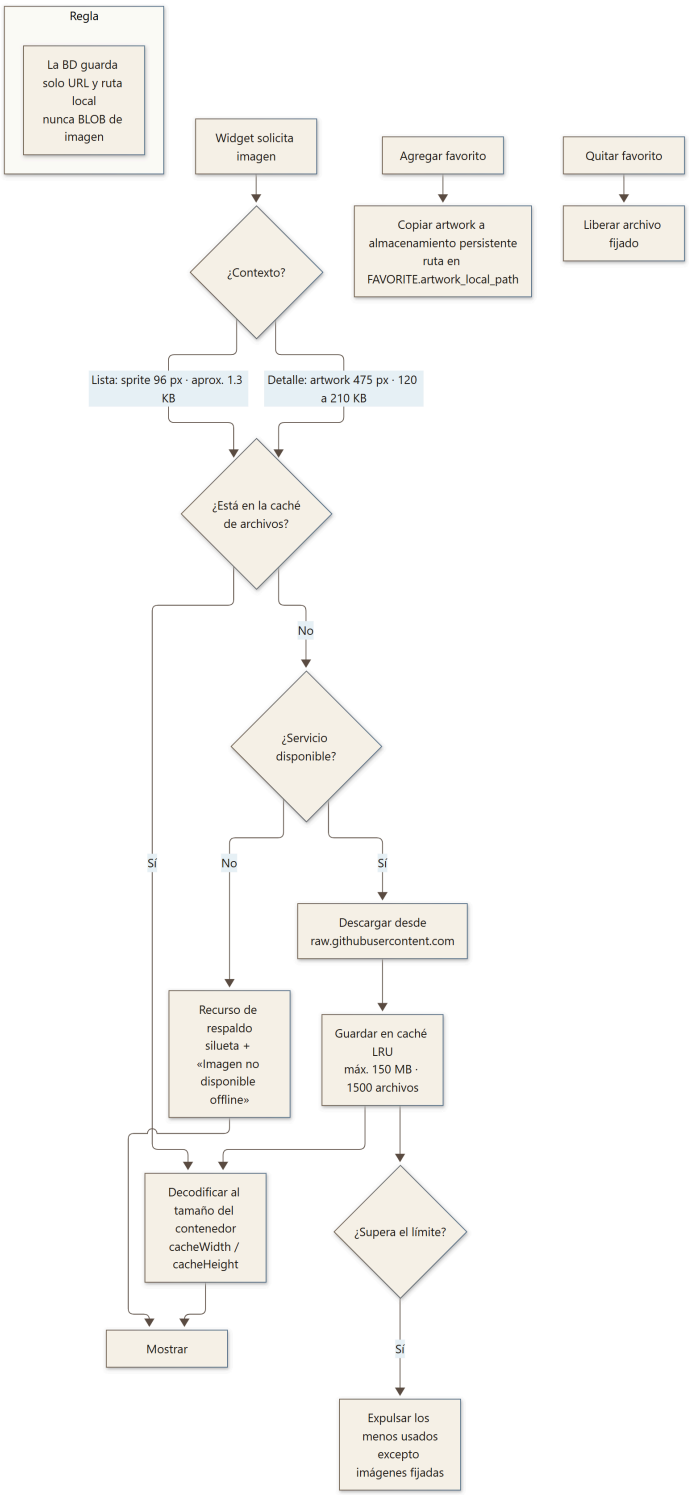


Figura 15. Política de imágenes (D15). La BD solo guarda URL y ruta local.

6. Integración GraphQL con PokéAPI

6.1 Configuración

PARÁMETRO	VALOR
Endpoint	`https://graphql.pokeapi.co/v1beta2`, solo POST y configurable por provider.
Esquema	Hasura v1beta2: tablas sin prefijo `pokemon_v2_` (`pokemon`, `pokemonspecies`, `pokemontypes` …). `pokemon_by_pk` **no existe**; usamos `pokemon(where: {id: {_eq: $id}})`.
Cliente	Paquete `graphql` con `HttpLink` y `FetchPolicy.noCache` (ADR-003).
Timeouts	5 s de conexión y 10 s de respuesta.
Paginación	`limit` / `offset` con `order_by: {id: asc}` y filtro `is_default: true`. La API no ofrece cursores, así que el offset es estable porque los ids no cambian.
Presupuesto	Máximo 90 llamadas por ventana móvil de 60 min, registradas en `api_call_log` (ADR-005).
Idiomas	Variable `$langs: ["es", "en"]` en nombres de tipos, grupos huevo, habilidades y especies.

Validamos las operaciones el 23-oct-2026 contra el servicio real (10 llamadas en total). Tiempos de 290 a 760 ms y respuestas `cache-control: max-age=1800`. El tipo 19 (*stellar*) existe en `type` pero lo excluimos de los filtros, porque solo aplica a la Teracristalización.

6.2 Catálogo de operaciones

OPERACIÓN	CUÁNDO	LLAMADAS	TAMAÑO
`HealthCheck`	Cambio de red, salida de «no disponible», sin tráfico por 10 min	1	31 B
`GetReferenceData`	Primera ejecución y cada 30 días	1	~147 KB
`GetPokemonPage`	Carga inicial y cada 7 días	6	~165 KB/lote
`GetPokemonsByIds`	Verificación OF-19/20	1 por lote	~1 KB/Pokémon
`SearchPokemonRemote`	Búsqueda sin resultados locales y con servicio	1	Pequeño
`GetPokemonDetail`	Abrir un detalle vencido o ausente	1	~110 KB (Charizard)

Fragmento compartido

```
# Fragmento compartido por GetPokemonPage, GetPokemonsByIds y SearchPokemonRemote.
fragment PokemonLight on pokemon {
  id
  name
  height
  weight
  pokemon_species_id
  pokemonspecy {
    generation_id
    evolution_chain_id
    is_legendary
    is_mythical
    pokemonegggroups { egg_group_id }
  }
  pokemontypes(order_by: { slot: asc }) { slot type_id }
  pokemonabilities(order_by: { slot: asc }) { ability_id is_hidden slot }
  pokemonstats { stat_id base_stat }
  pokemonsprites {
    front: sprites(path: "front_default")
    artwork: sprites(path: "other['official-artwork'].front_default")
  }
}
```

Página del catálogo

```
# Página del catálogo ligero (paginación offset/limit ordenada por id).
# Solo los campos que necesitan la lista, la búsqueda, los filtros y el orden.
query GetPokemonPage($limit: Int!, $offset: Int!) {
  pokemon(
    where: { is_default: { _eq: true } }
    order_by: { id: asc }
    limit: $limit
    offset: $offset
  ) {
    ...PokemonLight
  }
}
```

Detalle completo (una sola llamada, incluida la cadena evolutiva)

```
# Detalle completo bajo demanda en UNA llamada: identidad, stats, habilidades,
# movimientos (solo ids; los datos del movimiento vienen de GetReferenceData),
# especie, formas/variantes, sprites y cadena evolutiva.
query GetPokemonDetail($id: Int!, $langs: [String!]!) {
  pokemon(where: { id: { _eq: $id } }) {
    id
    name
    height
    weight
    is_default
    pokemontypes(order_by: { slot: asc }) { slot type_id }
    pokemonstats { stat_id base_stat }
    pokemonabilities(order_by: { slot: asc }) {
      is_hidden
      slot
      ability {
        id
        name
        abilitynames(where: { language: { name: { _in: $langs } } }) {
          name
          language { name }
        }
        abilityeffecttexts(where: { language: { name: { _in: $langs } } }) {
          short_effect
          language { name }
        }
      }
    }
    pokemonmoves {
      move_id
      level
      move_learn_method_id
      version_group_id
    }
    pokemonsprites {
      front: sprites(path: "front_default")
      front_shiny: sprites(path: "front_shiny")
      artwork: sprites(path: "other['official-artwork'].front_default")
      artwork_shiny: sprites(path: "other['official-artwork'].front_shiny")
    }
    pokemonspecy {
      id
      name
      generation_id
      is_legendary
      is_mythical
      pokemonegggroups { egg_group_id }
      pokemonspeciesnames(where: { language: { name: { _in: $langs } } }) {
        name
        genus
        language { name }
      }
      pokemons(order_by: { id: asc }) {
        id
        name
        is_default
        pokemonforms { form_name is_mega }
        pokemontypes(order_by: { slot: asc }) { slot type_id }
      }
```

```
      evolutionchain {
        id
        pokemonspecies(order_by: { order: asc }) {
          id
          name
          evolves_from_species_id
          pokemonevolutions {
            min_level
            min_happiness
            time_of_day
            evolutiontrigger { name }
            item { name }
          }
        }
      }
    }
  }
```

Datos de referencia

```
# Datos de referencia (1 sola llamada): tipos, generaciones, grupos huevo,
# tabla de eficacias, version groups, habilidades, movimientos y el total de
# Pokémon por defecto (para saber si el catálogo local está completo).
query GetReferenceData($langs: [String!]!) {
  type(where: { id: { _lt: 10000 } }, order_by: { id: asc }) {
    id
    name
    typenames(where: { language: { name: { _in: $langs } } }) {
      name
      language { name }
    }
  }
  generation(order_by: { id: asc }) {
    id
    name
    region { name }
  }
  egggroup(order_by: { id: asc }) {
    id
    name
    egggroupnames(where: { language: { name: { _in: $langs } } }) {
      name
      language { name }
    }
  }
  typeefficacy {
    damage_type_id
    target_type_id
    damage_factor
  }
  versiongroup(order_by: { order: asc }) {
    id
    name
    generation_id
  }
  ability(where: { is_main_series: { _eq: true } }, order_by: { id: asc }) {
    id
    name
  }
  move(order_by: { id: asc }) {
    id
    name
    type_id
    power
    accuracy
    pp
    priority
    move_damage_class_id
  }
  pokemon_aggregate(where: { is_default: { _eq: true } }) {
    aggregate { count }
  }
}
```

Verificación de integridad, búsqueda remota y salud

```
# Verificación de integridad (OF-19/20): trae la versión remota de un lote de
# filas del catálogo para compararla con la local y restaurarla si difiere.
query GetPokemonsByIds($ids: [Int!]!) {
  pokemon(where: { id: { _in: $ids } }, order_by: { id: asc }) {
    ...PokemonLight
  }
}
```

```
# Respaldo online de la búsqueda: solo se usa cuando el término no existe en la
# base local y el servicio está disponible. El resultado se persiste antes de
# mostrarse (la UI nunca consume esta respuesta directamente).
query SearchPokemonRemote($pattern: String!, $limit: Int!) {
  pokemon(
    where: { is_default: { _eq: true }, name: { _ilike: $pattern } }
    order_by: { id: asc }
    limit: $limit
  ) {
    ...PokemonLight
  }
}
```

```
# Comprobación de salud: confirma que el motor GraphQL y la base de datos de
# PokéAPI responden. Respuesta mínima (un id) para no gastar ancho de banda.
query HealthCheck {
  pokemon(limit: 1) {
    id
  }
}
```

6.3 Manejo de errores

ORIGEN	DETECCIÓN	`FAILURE` DEL DOMINIO	COMPORTAMIENTO
Sin red	`SocketException` sin interfaz	`NetworkFailure`	Offline; esperar evento de red.
Timeout	`TimeoutException`	`TimeoutFailure`	Servicio no disponible; backoff.
HTTP 429	Código de estado y `Retry-After`	`RateLimitFailure`	Pausar el presupuesto hasta `Retry-After` (mínimo 60 s).
HTTP 5xx o host caído	Código de estado	`ServiceUnavailableFailure`	Servicio no disponible; backoff.
`errors` sin `data`	Cuerpo GraphQL	`GraphQLFailure`	Registrar; no reintentar si es de validación.
`errors` con `data` parcial	Cuerpo GraphQL	`PartialDataFailure`	Persistir solo las entidades completas y validadas; reintentar el resto.
DTO inválido	Validación del mapeo	`ValidationFailure`	Descartar el lote sin tocar la BD (OF-04).

7. Integración con Pokémon Champions Battle Data

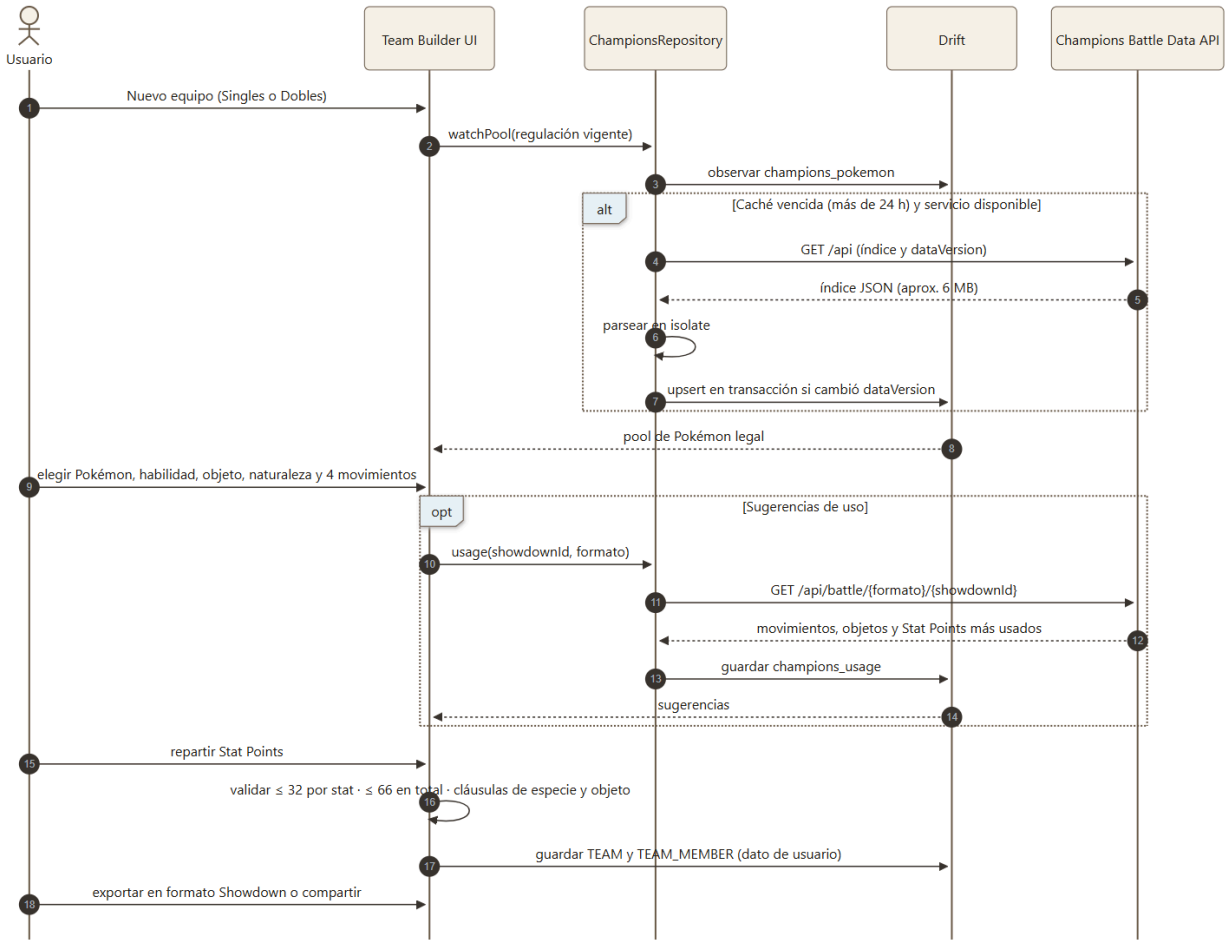


Figura 16. Datos de Champions y construcción de equipos (D16).

ENDPOINT	USO	FRECUENCIA
`GET /api`	Índice (~6 MB): `dataVersion`, `defaultSeason`, temporadas y, por Pokémon, `showdownId`, tipos, habilidades, stats base, sprite y `learnableMoveNames`.	Una vez cada 24 h; se reemplaza solo si cambia `dataVersion`.
`GET /api/battle/{Singles|Doubles}/{showdownId}`	Filas de uso por categoría: `move`, `held_item`, `ability`, `teammate`, `stat_alignment` y `stat_points` (reparto de SP con porcentaje).	Bajo demanda al editar un miembro; caché de 24 h por Pokémon y formato.
`GET /api/metadata/{nombre}`	Formas de un Pokémon base.	Solo si el índice no trae la forma.

Decisiones.

- **Pool legal:** los Pokémon presentes en el índice para `defaultSeason`.

- **Reglas:** el servicio no publica la regulación, así que la empaquetamos en `assets/champions/regulation.json`, versionado con la app. Los valores marcados se confirman contra la regulación oficial vigente al implementar.
- **Mapeo a PokéAPI:** por número de dex y nombre de forma (`raichualola` ↔ `raichu-alola`). Sirve para reutilizar artwork, tipos y datos de movimientos.
- PokéAPI ya incluye el version group `champions` **(id 32)**, que usamos para validar `learnableMoveNames`.

```
{
  "id": "M-B",
  "level": 50,
  "teamSize": 6,
  "statPoints": { "perStat": 32, "total": 66 },
  "clauses": ["species", "item"],
  "bring": { "singles": 3, "doubles": 4 },
  "notes": "bring y cláusulas: confirmar con la regulación oficial vigente"
}
```

Validación de Stat Points. Para cada estadística `0 ≤ sp ≤ 32` y `Σ sp ≤ 66`. La estadística final a nivel 50 se calcula con la base, los SP (cada SP suma 1 punto a nivel 50) y la naturaleza (×1.1 / ×0.9).

Exportación. Texto compatible con Pokémon Showdown (`Nombre @ Objeto`, `Ability:`, `Level: 50`, naturaleza y `- Movimiento`). La línea de SP se ajusta al formato que acepte Showdown para Champions.

Atribución. El pie del Team Builder muestra «Battle data provided by Pokémon Champions Battle Data» con enlace. No redistribuimos el índice.

8. Sistema de diseño Regi

Base: **Material 3** (`useMaterial3: true`) con un `ColorScheme` generado por tema y una `ThemeExtension<RegiTheme>` con los tokens propios de la Pokédex: `shell` , `onShell` , `screen` , `onScreen` , `accent` y los colores del LED.

Los componentes Pokédex son widgets propios en `lib/ui/components` :

- `PokedexShell` : cabecera con lente-LED, puntos braille de los Regis, borde dentado y barra inferior.
- `DexScreen` : pantalla con bisel y esquina inferior redondeada.
- `StatusLed` : glifo, anillo de sync y popover.
- `TypeChip` y `StatBar` .

TEMA	CONCEPTO	CLARO: CARCASA / PRIMARIO / ACENTO	OSCURO: CARCASA / PRIMARIO / ACENTO	CONTRASTE MÍN. CLARO	CONTRASTE MÍN. OSCURO
Regirock	Roca y arcilla; puntos naranjas	`#8E4F2B` / `#8A4B25` / `#C46A12`	`#5E3418` / `#E6A673` / `#F2A35A`	6.4:1	8.3:1
Regice	Hielo cristalino; brillo celeste	`#5BB4E5` / `#1E6CA4` / `#1580B8`	`#1C4E6E` / `#8FD3F4` / `#6FC8EE`	5.6:1	7.9:1
Registeel	Acero pulido; puntos rojos	`#5F6B77` / `#4A5561` / `#C93838`	`#39424B` / `#B9C3CD` / `#EE6B6B`	5.4:1	9.1:1
Regigigas	Coloso blanco; bandas doradas y musgo	`#EDE5CF` / `#7E6210` / `#5E8C3A`	`#4A4130` / `#E2C25A` / `#8DBF63`	5.8:1	8.7:1
Regieleki	Electricidad; amarillo, negro y cian	`#F5C800` / `#1F1F1F` / `#0E8FA1`	`#2A2A2A` / `#FFD84D` / `#3FD6E8`	11.5:1	10.4:1
Regidrago	Dragón ancestral; granate y oro	`#9C2F2F` / `#8E2A2A` / `#A8720F`	`#5C1C1C` / `#F09A8A` / `#E7B04A`	7.3:1	7.8:1

El script `docs/tools/contrast.mjs` verifica que todos los pares texto/fondo superen 4.5:1 y que los acentos no textuales superen 3:1, en claro y en oscuro.

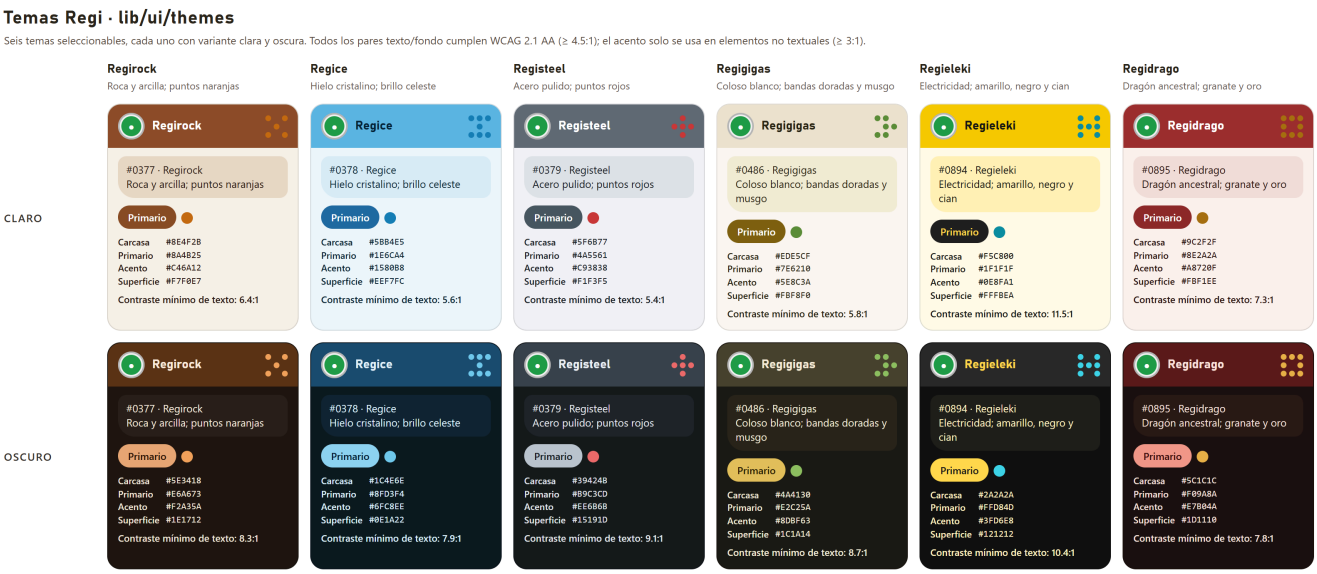


Figura 17. Los seis temas Regi (M02).

9. Navegación

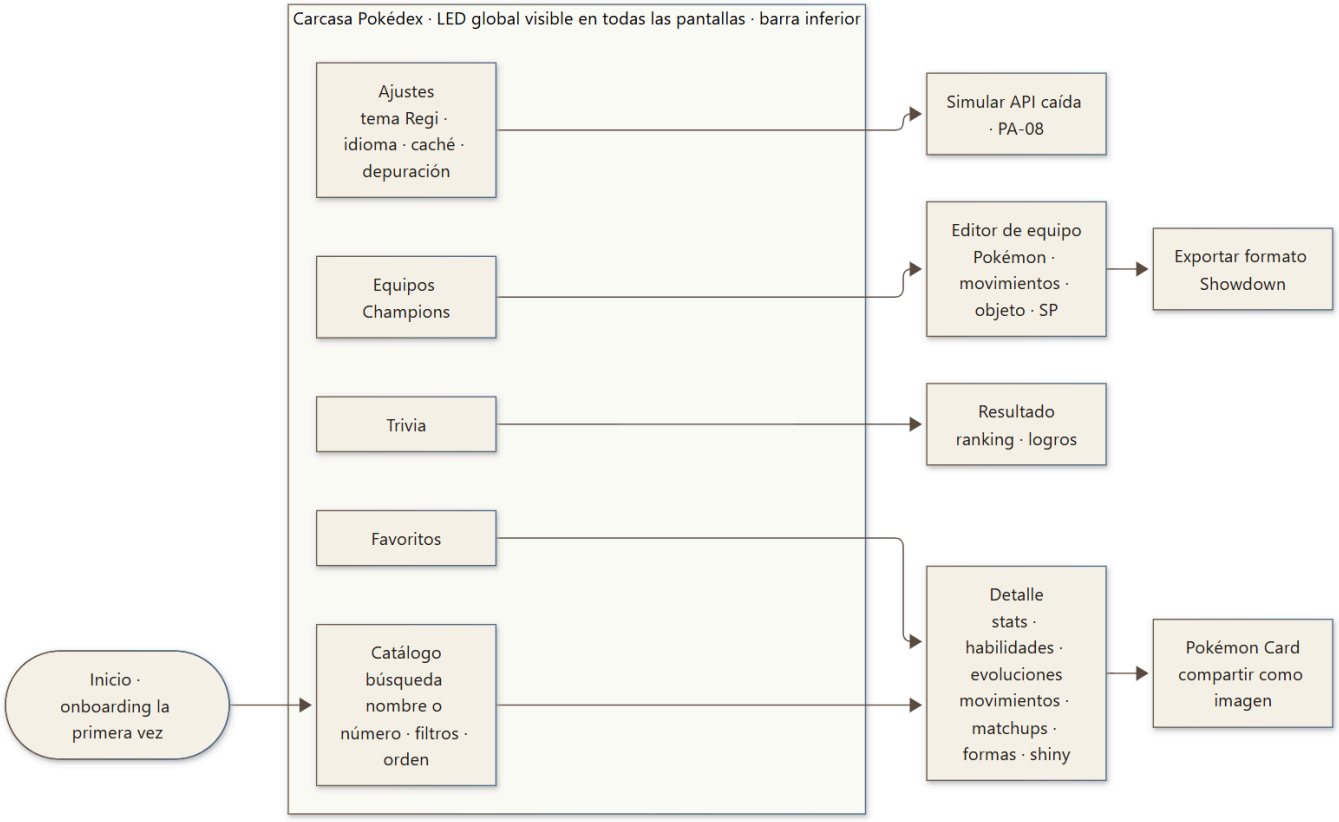


Figura 18. Mapa de pantallas (D17). Usamos `go_router` *con* `StatefulShellRoute` *para conservar el estado de cada pestaña.*

10. Dependencias principales y su justificación

PAQUETE	USO	JUSTIFICACIÓN
`flutter_riverpod`	Estado e inyección de dependencias	Providers testeables y sustituibles; `StreamProvider` encaja con los streams de Drift (ADR-002).
`drift`, `drift_flutter`, `sqlite3_flutter_libs`	Persistencia	SQL tipado, consultas reactivas, transacciones y migraciones (ADR-001).
`graphql`	Cliente GraphQL	Cliente Dart puro con links y control de políticas de caché (ADR-003).
`connectivity_plus`	Eventos de red	Pista para disparar el health-check (ADR-007).
`cached_network_image`, `flutter_cache_manager`	Imágenes	Caché de disco acotada y LRU (ADR-004).
`path_provider`	Rutas	Directorio para las imágenes fijadas de favoritos.
`go_router`	Navegación	Rutas declarativas y shell con pestañas.
`freezed`, `json_serializable`	Modelos	Entidades inmutables, estados sellados y DTOs.
`crypto`	Hash	`content_hash` SHA-1 para OF-19 (ADR-006).
`fl_chart`	Gráficos	Barras o radar de estadísticas (RF-DET-02).
`share_plus`	Compartir	Pokémon Card y texto Showdown.
`flutter_localizations`, `intl`	i18n	ARB es/en (ADR-011).
`mocktail` (dev)	Pruebas	Dobles de repositorios y fuentes de datos.

Las versiones exactas se fijan en `pubspec.yaml` al iniciar el Sprint 1.

11. Architecture Decision Records

ADR-001 · Drift (SQLite) como fuente de verdad observable

- **Estado:** Aceptado (23-oct-2026) · **Responsable:** Randy
- **Contexto:** OF-16 exige una persistencia estructurada y consultable, con búsquedas y filtros offline, transacciones y migraciones.
- **Decisión:** Drift sobre SQLite. Los DAOs exponen `Stream`s que la UI observa por medio de los repositorios.
- **Alternativas:** Isar o Hive (NoSQL, filtros relacionales más torpes, futuro de mantenimiento incierto); `sqflite` puro (sin tipos ni streams); caché del cliente GraphQL (no cumple OF-16).
- **Consecuencias:** Requiere generación de código (`build_runner`) y diseño de esquema. A cambio da SQL verificable, índices, transacciones (RNF-RES-04) y compatibilidad con el Database Inspector.

ADR-002 · Riverpod para estado e inyección de dependencias

- **Estado:** Aceptado · **Responsable:** Rafael
- **Contexto:** Necesitamos estados explícitos (carga, éxito, vacío y error), dependencias sustituibles y poco boilerplate entre dos personas.
- **Decisión:** `flutter_riverpod`. Los providers raíz en `app/` construyen el grafo; las features exponen `Notifier` / `StreamProvider`.
- **Alternativas:** BLoC (más ceremonia por feature); `get_it` + `ChangeNotifier` (estados implícitos).
- **Consecuencias:** El override de providers simplifica las pruebas y el simulador de caída de PA-08. Hay que cuidar el ciclo de vida (`autoDispose`).

ADR-003 · Cliente `graphql` sin caché normalizada y DTOs propios

- **Estado:** Aceptado · **Responsable:** Randy
- **Contexto:** La fuente de verdad es Drift; una caché adicional del cliente duplicaría datos y confundiría el origen.
- **Decisión:** Paquete `graphql` con `FetchPolicy.noCache`. Las operaciones viven en archivos `.graphql` y los DTOs se mapean manualmente a filas y entidades.
- **Alternativas:** `ferry` con codegen (tipado fuerte pero pesado para 6 operaciones); `http` con JSON a mano (sin manejo estándar de `errors`).
- **Consecuencias:** Mapeo manual que hay que probar, a cambio de control total sobre las respuestas parciales (RNF-RES-05).

ADR-004 · Imágenes fuera de la base de datos

- **Estado:** Aceptado · **Responsable:** Rafael
- **Contexto:** El artwork oficial pesa entre 120 y 210 KB por Pokémon (~165 MB para 1025). Guardarlo como BLOB inflaría SQLite, ralentizaría las consultas y las copias, y dificultaría la limpieza. Los sprites pesan ~1.3 KB.
- **Decisión:**
  - La BD solo guarda URL y ruta local.
  - Las imágenes van en una caché de archivos LRU de 150 MB y 1500 archivos.

  - Las listas usan sprites y el detalle usa artwork.
  - El artwork de los favoritos se copia a `pinned/` y no se expulsa.
  - Se decodifica al tamaño del contenedor.

- **Alternativas:** BLOB en SQLite; descargar todo el artwork al inicio (consumo de datos y espacio inaceptable); depender de la caché HTTP (no garantiza imágenes offline).
- **Consecuencias:** Una imagen puede no estar disponible offline, así que se muestra el respaldo (OF-08). Los favoritos siempre tienen imagen.

ADR-005 · Presupuesto de llamadas a PokéAPI

- **Estado:** Aceptado · **Responsable:** Randy
- **Contexto:** PokéAPI GraphQL permite 100 llamadas por hora por IP y banea a quien abusa. La app hace health-checks, carga inicial, detalle y verificación de integridad.
- **Decisión:**
  - Un `ApiBudget` con ventana móvil de 60 min y tope de **90** llamadas.
  - Lotes de 200 y un solo request por detalle (incluida la cadena evolutiva).
  - Las consultas exitosas cuentan como health-check.
  - Ante un 429 se pausa según `Retry-After`.
  - Prioridad: interacción del usuario > integridad > refresco periódico.
- **Alternativas:** Sin control (riesgo de ban en plena demo); REST de PokéAPI (no cumple el requisito de GraphQL).
- **Consecuencias:** El refresco periódico puede posponerse. La carga inicial completa cuesta 7 llamadas.

ADR-006 · Detección de alteraciones locales con hash de contenido

- **Estado:** Aceptado · **Responsable:** Randy
- **Contexto:** OF-19 exige detectar y restaurar datos del catálogo editados en la BD local (PA-16) sin romper los datos del usuario.
- **Decisión:** `content_hash` SHA-1 por fila al persistir, recálculo local en cada disparador y verificación rotativa contra el remoto en lotes de 200, más comparación completa al abrir un detalle. El remoto gana.
- **Alternativas:** Sobrescribir todo en cada sync (gasta el presupuesto); comparar solo `updated_at` (no detecta ediciones manuales).
- **Consecuencias:** El costo de CPU es despreciable. Una edición que también recalcule el hash solo se detecta en la verificación remota, lo cual es aceptable.

ADR-007 · Monitor de conectividad con health-check e histéresis

- **Estado:** Aceptado · **Responsable:** Randy
- **Contexto:** IND-02 prohíbe inferir «online» solo por Wi-Fi o datos; IND-06 prohíbe el parpadeo.
- **Decisión:** `connectivity_plus` como disparador más `HealthCheck` como confirmación. Estados de servicio y de sync separados, e histéresis de 1.5 s.
- **Alternativas:** `internet_connection_checker` (comprueba Internet genérico, no PokéAPI); ping a DNS público.

- **Consecuencias:** El estado refleja PokéAPI de verdad, y detectar la caída tarda hasta 5 s más la histéresis.

ADR-008 · Champions Battle Data como fuente secundaria

- **Estado:** Aceptado · **Responsable:** Randy
- **Contexto:** El Team Builder necesita el pool legal, los movimientos aprendibles, el uso y los repartos de SP de Pokémon Champions. PokéAPI no publica uso ni regulación.
- **Decisión:**
  - Consumir el índice y los datos de batalla de championsbattledata.com con caché de 24 h en tablas propias (D12).
  - Reglas en `regulation.json` empaquetado y atribución visible.
  - Usar el version group `champions` de PokéAPI como validación cruzada.
- **Alternativas:** Cargar los datos a mano (se desactualizan); scraping (viola sus reglas).
- **Consecuencias:** Dependemos de un servicio comunitario sin SLA. Si falla, el Team Builder sigue funcionando con la caché. Hay que respetar sus reglas de uso.

ADR-009 · Sistema de diseño Regi sobre Material 3

- **Estado:** Aceptado · **Responsable:** Rafael
- **Contexto:** Queremos una identidad propia (Pokédex moderna y seis temas Regi) sin perder accesibilidad ni consistencia.
- **Decisión:** Material 3 con `ThemeExtension<RegiTheme>` y componentes propios en `lib/ui/`. Los tokens salen de una única tabla de paletas verificada por script.
- **Alternativas:** `shadcn_ui` o `forui` (más dependencias y más trabajo de re-estilizado).
- **Consecuencias:** Más widgets propios que mantener, pero sin dependencias extra y con el soporte de accesibilidad de Material.

ADR-010 · Simulador de caída de API e inspector de BD

- **Estado:** Aceptado · **Responsable:** Rafael
- **Contexto:** PA-08 y PA-16 deben demostrarse en vivo y de forma reproducible.
- **Decisión:** Interruptor de depuración que redirige el endpoint a una ruta inválida con la red activa, más el Database Inspector de Android Studio.
- **Alternativas:** Proxy externo (frágil en clase); desconectar el Wi-Fi (no reproduce «red sí, API no»).
- **Consecuencias:** El código de depuración queda aislado en `settings/debug` y no se incluye en release.

ADR-011 · Internacionalización con ARB (es/en)

- **Estado:** Aceptado · **Responsable:** Rafael
- **Contexto:** RF-TRV-06 exige la trivia en español e inglés; RF-COM-05 recomienda toda la app.
- **Decisión:** `flutter_localizations` + `intl` con `app_es.arb` y `app_en.arb`. Los nombres de PokéAPI se guardan en ambos idiomas.
- **Alternativas:** Paquetes de traducción en tiempo de ejecución (innecesarios).
- **Consecuencias:** Toda la UI pasa por `AppLocalizations`. Los efectos de habilidades sin traducción en PokéAPI se muestran en inglés.

12. Riesgos y puntos críticos

RIESGO	PROB.	IMPACTO	MITIGACIÓN
Límite de 100 llamadas/h o ban de IP en la demo	Media	Alto	ADR-005, dispositivo pre-sincronizado y registro de llamadas.
Caída diaria de PokéAPI (01:00 UTC = 21:00 RD)	Alta	Medio	Se muestra como «Servicio no disponible»; no ensayamos a esa hora.
Cambios del esquema v1beta2	Baja	Alto	Consultas aisladas en `.graphql` y validación al inicio de cada sprint.
Champions Battle Data cambia o cae	Media	Medio	Caché de 24 h, reglas locales y Team Builder degradado pero usable.
Espacio por imágenes	Media	Medio	ADR-004 y botón de limpieza.
Sin Mac para iOS	Media	Medio	Demo en Android; código sin dependencias nativas propias.
Aula sin Internet	Media	Alto	Datos pre-sincronizados; los escenarios con red se hacen con hotspot.
Defensa eliminatoria	—	Crítico	Revisión cruzada de PR y ensayo explicando el código del compañero.

13. Cronograma

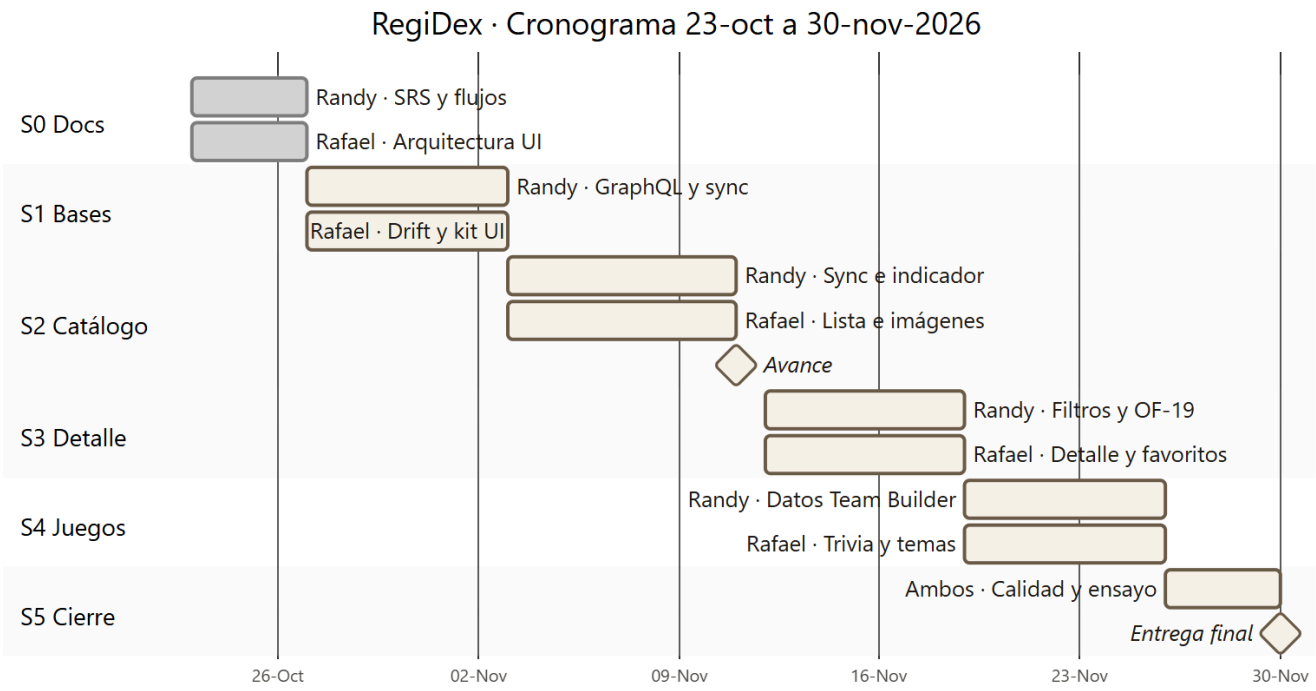


Figura 19. Cronograma del 23-oct al 30-nov-2026 (D18). El Avance se entrega el 11-nov-2026.